

Lectura por consola:

```
import java.util.*;
Scanner sc = new Scanner(System.in);
String s = sc.nextLine();      // línea completa
int n = sc.nextInt();         // int (ojo: deja \n pendiente)
double d = sc.nextDouble();   // double
boolean b = sc.nextBoolean(); // true/false
// Truco tras nextInt/nextDouble:
sc.nextLine(); // consumir salto de línea antes de pedir texto
//Comprobar si lo tecleado es número (sin excepciones):
if (sc.hasNextInt()) { int x = sc.nextInt(); }
else { sc.nextLine(); /* limpiar */ }
```

```
import java.io.*;

BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

String linea = br.readLine();      //
siempre String
int n = Integer.parseInt(linea);
double d = Double.parseDouble(br.readLine());
char c = br.readLine().charAt(0);
```

Salida por consola:

```
%d entero (int, long) %f real (double, float) %s String %c char %b boolean %% símbolo %
System.out.printf("Media: %.2f\n", 3.14159); // 3.14
System.out.printf("|%10s|\n", "Ana"); // ancho 10 derecha
System.out.printf("|%-10s|\n", "Ana"); // ancho 10 izquierda
System.out.printf("%05d\n", 42); // 00042
```

Métodos más usados

```
s.length()
s.charAt(i)
s.substring(a, b)
s.equals(t) // comparar contenido
s.equalsIgnoreCase(t)
s.toUpperCase()
s.toLowerCase()
s.trim()
s.replace("a","b")
s.contains("abc")
s.startsWith("pre")
s.endsWith("fin")
s.indexOf("x") // -1 si no
s.split(",") // devuelve String[]
```

```
Math.abs(x)
Math.max(a,b)
Math.min(a,b)
Math.pow(a,b)
Math.sqrt(x)
Math.round(x)
```

```
java.util.Arrays.toString(arr)
java.util.Arrays.sort(arr)
java.util.Arrays.equals(a,b)
```

```
int n = Integer.parseInt("123");
double d = Double.parseDouble("3.14")
```

POO

<pre>public class Persona { private String nombre; // atributos SIEMPRE private private int edad; public Persona(String nombre, int edad) { // constructor this.nombre = nombre; this.edad = edad; } public String getNombre() { return nombre; } public void setEdad(int edad) { this.edad = edad; } @Override public String toString() { return nombre + " - " + edad; } } this → objeto actual Encapsulamiento = atributos private + getters/setters</pre>	STATIC (DE CLASE) <pre>private static int contador; public static int getContador() { return contador; }</pre> Pertenece a la clase Se llama: Clase.metodo() Método static SOLO usa cosas static	HERENCIA <pre>public class Empleado extends Persona { private double sueldo; public Empleado(String nombre, int edad, double sueldo) { super(nombre, edad); // obligatorio si no hay constructor vacío this.sueldo = sueldo; } }</pre> extends = herencia simple Se heredan atributos y métodos públicos/protected super() siempre primera línea ----- ----- this: atributo objeto actual super: método de superclase
CREAR OBJETOS <pre>Persona p = new Persona("Ana", 20);</pre> Sin new no hay objeto (solo referencia).	CLASE ABSTRACTA <pre>public abstract class Animal{ public abstract void hacerSonido(); }</pre> Subclase: <pre>public class Perro extends Animal{ @Override public void hacerSonido(){ System.out.println("Guau"); } }</pre> No se puede hacer new Animal()	INTERFACES <pre>public interface Volador { void volar(); }</pre> <pre>public class Pajaro implements Volador { public void volar() { } }</pre> Una clase puede implementar varias interfaces Métodos obligatorios
ENUM -Básico <pre>public enum Estado { ACTIVO, INACTIVO, SUSPENDIDO }</pre> -Uso: <pre>Estado e = Estado.ACTIVO;</pre> -Enum con atributos <pre>public enum Nivel { BAJO(1), MEDIO(2), ALTO(3); private int valor; Nivel(int valor) { this.valor = valor; } public int getValor() { return valor; }</pre>		RELACIONES ENTRE CLASES -Asociación <pre>private Departamento departamento;</pre> Asociación → relación -Composición <pre>private Motor motor;</pre> Composición → parte-de fuerte